ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Luma Wallet — веб-платформа криптоплатежей с интеграцией MetaMask

Версия: 2.0 · Дата: 8 ноября 2026 · Формат: учебный прототип

1. Общая информация

Цель: разработать доступный из браузера учебный веб-интерфейс для работы с Ethereum-кошельком: подключение, просмотр баланса, отправка ETH, получение адреса и формирование запроса на оплату. Деморежим позволяет демонстрировать сценарии без реальных средств.

Тип системы: некастодиальное Web3-приложение (frontend + подключаемый провайдер MetaMask). Сервис не хранит приватные ключи и не является банком или платёжным оператором.

2. Роли и пользователи

• Пользователь — просмотр баланса, выбор режима, отправка и получение ETH, история.

• Плательщик — открывает ссылку запроса оплаты, проверяет реквизиты и подтверждает перевод через MetaMask.

• MetaMask — внешняя система для согласия пользователя и подписи транзакций.

3. Функциональные требования

ID	Требование
FR-01	Просматривать обзор, баланс и сеть Ethereum / Sepolia.
FR-02	Подключать MetaMask с явным разрешением пользователя.
FR-03	Проверять Ethereum-адрес и сумму с точностью до 18 знаков.
FR-04	Создавать отправку ETH через EIP-1193 с подтверждением MetaMask.
FR-05	Отображать адрес получения, QR и ссылку на оплату.
FR-06	Формировать ссылку запроса оплаты с суммой, сетью и примечанием.
FR-07	Хранить локальную историю исходящих транзакций и обновлять их статусы.
FR-08	Работать в автономном учебном режиме без реальных переводов.
FR-09	Поддерживать мобильную и настольную версии.
FR-10	Показывать сообщения при отмене, недостатке средств и неверной сети.

4. Нефункциональные требования

• NFR-01: публикация по HTTPS в облаке (Render Static Site) после подключения аккаунта хостинга.

• NFR-02: без запроса и хранения seed-фраз, паролей и приватных ключей.

• NFR-03: отзывчивый интерфейс от ширины 320 px.

• NFR-04: данные демонстрации сохраняются локально в браузере без сервера.

• NFR-05: бизнес-логика разделена на независимые сервисы (SOLID).

• NFR-06: есть автоматические тесты критических правил проверки, записи истории и платежей.

5. Технологический стек

Frontend: HTML5, CSS3, JavaScript ES2022. Wallet adapter: EIP-1193 JSON-RPC (MetaMask). Хранение: localStorage. QR: локальная MIT-библиотека. Тесты: Node.js node:test и браузерные smoke tests. Хостинг: Render Static Site, HTTPS/CDN. Документация: PDF и UML/C4.

6. Сценарии и потоки

UC-01 — Реальная отправка ETH

1) Пользователь подключает MetaMask. 2) Приложение получает разрешённый адрес, сеть и баланс. 3) Пользователь заполняет реквизиты. 4) Валидатор проверяет адрес, сумму, доступный баланс и сеть. 5) Приложение вызывает eth_sendTransaction. 6) MetaMask показывает окно подтверждения. 7) После одобрения возвращается hash и появляется статус Pending. 8) Приложение запрашивает receipt и показывает Success или Failed.

UC-02 — Учебная отправка

Пользователь выбирает Demo, вводит адрес и сумму, подтверждает действие. DemoPaymentProcessor проверяет баланс, уменьшает учебное значение и сохраняет учебную запись; блокчейн не вызывается.

UC-03 — Запрос оплаты

Получатель подключает MetaMask, вводит сумму и назначение на экране «Получить», копирует ссылку. Плательщик открывает ссылку: адрес, сеть и сумма автоматически подставляются в форму. Фактическое списание требует его подтверждения в MetaMask.

7. Данные

Transaction: id, hash (только в реальном режиме), type, amount, note, to, time, status. Локальная история организована по адресу и сети. Демо хранится под отдельными ключами; приватные ключи не передаются приложению.

8. Архитектура и SOLID

• S — Single Responsibility: JsonStorage хранит JSON; PaymentValidator валидирует; TransactionRepository управляет историей.

• O — Open/Closed: новый кошелёк можно поддержать через другой gateway, не меняя процессор платежей.

• L — Liskov Substitution: DemoPaymentProcessor и EthereumPaymentProcessor наследуют PaymentProcessor и асинхронно возвращают унифицированный объект {mode, transaction, hash, status}.

• I — Interface Segregation: процессор вызывает только sendNative() и receipt(), а репозиторий — list()/prepend()/save(); лишние функции не требуются.

• D — Dependency Inversion: EthereumPaymentProcessor получает gateway и repository в конструкторе, а не создаёт их внутри. В тестах зависимости заменяются заглушками.

9. Безопасность и ограничения

• Для демонстрации использовать Sepolia Testnet и тестовый ETH. Mainnet списывает реальные средства, переводы необратимы.

• Ссылка на платёж — это предзаполнение формы, а не автоматически оплаченный инвойс или подтверждённый чек.

• Без отдельного индексатора история не отражает все входящие транзакции блокчейна.

• Продукт учебный, не содержит серверной бухгалтерии, коммерческой обработки заказов или KYC/AML.

• Статус облачной публикации: конфигурация подготовлена; фактический публичный URL появляется после развертывания в аккаунте Render.

10. Критерии приёмки

• Открытие index.html показывает Demo без регистрации; вкладки и адаптивная вёрстка работают.

• После учебного платежа баланс уменьшается, операция появляется в истории.

• При отсутствии MetaMask интерфейс предлагает его установить и не отправляет реальную транзакцию.

• При подключённом MetaMask отображаются адрес, сеть и баланс, реальные платежи подтверждаются в кошельке.

• В пакете находятся C4, Use Case, Activity, Class, Sequence, State и Deployment диаграммы.

• Каждый принцип SOLID сопоставлен с кодом; tests/core.test.js проходит на Node.js.

• Для требования облачного сервера необходимо добавить действующий URL опубликованного сайта.

11. Состав сдачи

Корень проекта: index.html, styles.css, app.js, wallet-core.js, qr-vendor.js, favicon.svg, render.yaml. Документы: docs/Technical_Specification.pdf, docs/Diagrams.pdf, docs/SOLID_Explanation.md. Исходники схем: diagrams/*.dot и diagrams/*.svg. Тесты:

tests/core.test.js. Инструкция: README.md, DEPLOY_RENDER.md.

Литература: EIP-1193 (eips.ethereum.org/EIPS/eip-1193), MetaMask developer documentation (docs.metamask.io), Render Static Sites (render.com/docs/static-sites).